

Software Architecture. Interfaces exercise:

With this exercise, we create a program that sorts people by surname and name, then reuses the same sorting algorithm for rectangles ordered by their area. The code is organized in three packages that follow the three stages of the exercise: v1 contains the initial design, v2 introduces a custom interface, and v3 uses the standard Java Comparable interface with Collections.sort.

1. *Implement a class Person that:*
 - a. *Stores both name and surname.*
 - b. *Has a method to print the name and surname to the screen.*

Person stores the name and surname as private String fields. The constructor receives both values, two getters give read access, and print writes the full name to the console.

```
public void print() {  
    System.out.println(name + " " + surname);  
}
```

2. *Implement a class Sorter that it is able to sort an array of Persons (Person[]). It must:*
 - a. *Have a method that receives an array of Person*
 - b. *Sort the array by surname, name.*
 - c. *Use a simple algorithm like:*
 - i. https://en.wikipedia.org/wiki/Bubble_sort
 - ii. https://en.wikipedia.org/wiki/Insertion_sort

Sorter.sort(Person[] people) uses bubble sort. Each pass compares neighbouring elements and swaps them when the left one should go after the right one. The comparison is done by a private helper: surnames are compared first, and names are only compared when the surnames are equal.

```
private boolean isGreater(Person a, Person b) {  
    int c = a.getSurname().compareTo(b.getSurname());  
    if (c != 0) {  
        return c > 0;  
    }  
    return a.getName().compareTo(b.getName()) > 0;  
}
```

The inner loop runs up to $n - 1 - i$ because after each pass the largest remaining element is already at the end of the array. Equal elements are never swapped, so their original order is kept.

3. *Implement a Program that uses the Sorter to sort an array of 5 persons.*

v1.Main creates five people, calls sorter.sort(people) and prints the result. Two people share the surname Puig, which checks that the name is used as the second sorting key.

```
Laia Font  
Joan Puig  
Marc Puig  
Anna Serra  
Pau Vila
```

4. *Implement a class Rectangle that:*
 - a. *Stores its width and the height*
 - b. *Has a method to calculate its area*

Rectangle stores its width and height as private double fields. The constructor sets both dimensions and area returns their product. A rectangle of width 3 and height 4 returns 12.0.

```
public double area() {  
    return width * height;  
}
```

5. *Draw a diagram of the current classes of the system*

Main creates the objects and calls Sorter. Sorter depends directly on Person, because both its parameter type and its comparison logic use that class. Rectangle has no relation with Sorter, so in this design it cannot be sorted.

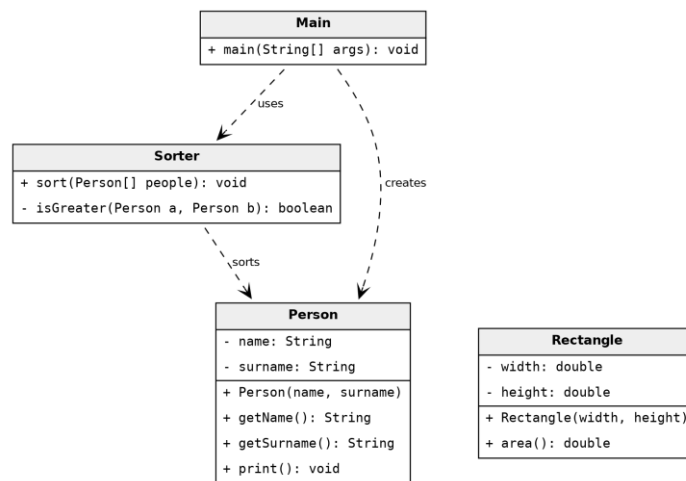


Figure 1

6. *Which modifications you have to do to make the Sorter class capable of sorting both Rectangles and Persons?*
 - a. *Rectangles must be sorted by its area*

The traversal and swapping steps of bubble sort do not depend on the type of object. Only the comparison does. The design therefore needs three changes:

- a) Create an interface, Sortable, with one method that compares two objects and returns a boolean.
- b) Make Person and Rectangle implement Sortable. Person compares by surname and then name; Rectangle compares by area.
- c) Change the parameter of Sorter.sort from Person[] to Sortable[] and call the interface method instead of the private helper.

- b. *Which element of your design enables the class Sorter and its sort method to accept both Persons and Rectangles?*

The Sortable interface. Sorter only knows that every element has an isBigger method; it does not know whether the elements are people or rectangles. Each class supplies its own ordering rule, and Sorter works with any class that implements the interface, including classes added in the future.

- c. Draw a new diagram with the proposed changes before trying to implement it.

Person and Rectangle implement Sortable, and Sorter depends only on the interface. The direct dependency between Sorter and Person from Figure 1 disappears.

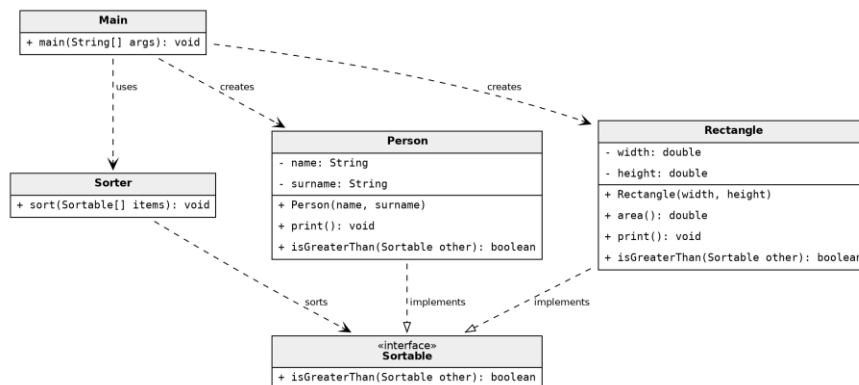


Figure 2

7. This element must have a method that receives two objects and return a boolean indicating if the second is bigger or not (in sort order) than the the first object. Make sure that your design follows this tip.

```

public interface Sortable {
    // true if "other" (2nd object) is bigger than this (1st object) in sort order
    boolean isBigger(Sortable other);
}

```

The method follows the tip of the exercise. The first object is the one that receives the call (this) and the second object is the argument (other). `isBigger` returns true when the second object is bigger than the first one in sort order, and false when it is smaller or equal.

This is the same structure as Java's `Comparable.compareTo`: one method, implemented by the class being sorted, that compares the current object with another one.

8. Implement your design and make a program that sorts an array of 5 Persons and other array of 5 Rectangles.

Each class implements `isBigger` with its own rule. The argument is cast to the concrete class, because each array only contains objects of one type.

```

// In Person
@Override
public boolean isBigger(Sortable other) {
    Person o = (Person) other;
    int c = o.surname.compareTo(surname);
    if (c != 0) {
        return c > 0;
    }
    return o.name.compareTo(name) > 0;
}

// In Rectangle
@Override
public boolean isBigger(Sortable other) {
    Rectangle o = (Rectangle) other;
    return o.area() > this.area();
}

```

Mariona Montal Morta

The bubble sort in Sorter is the same as in v1. Only the parameter type and the comparison call change:

```
public void sort(Sortable[] items) {
    int n = items.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (items[j + 1].isBigger(items[j])) {
                Sortable tmp = items[j];
                items[j] = items[j + 1];
                items[j + 1] = tmp;
            }
        }
    }
}
```

Bubble sort must swap two neighbours when the left one is bigger than the right one. With `isBigger` this question is asked from the right element: `items[j + 1]` is the first object and `items[j]` is the second, so the call returns true exactly when the left element is bigger. For example, with areas 12 and 4 next to each other, the call asks whether 12 is bigger than 4 and the two rectangles are swapped. Equal elements return false and are never swapped.

v2.Main uses one Sorter object to sort an array of five people and an array of five rectangles. Output:

```
Persons:
Laia Font
Joan Puig
Marc Puig
Anna Serra
Pau Vila
Rectangles:
1.0 x 1.0 -> area 1.0
2.0 x 2.0 -> area 4.0
5.0 x 2.0 -> area 10.0
3.0 x 4.0 -> area 12.0
6.0 x 3.0 -> area 18.0
```

9. *Your solution should be similar to the one, already present in standard Java. Have a look at the following documents:*

- a. <https://docs.oracle.com/javase/7/docs/api/java/lang/Comparable.html>
- b. [https://docs.oracle.com/javase/7/docs/api/java/util/Collections.html#sort\(java.util.List\)](https://docs.oracle.com/javase/7/docs/api/java/util/Collections.html#sort(java.util.List))

The Sortable interface follows the same idea as Java's Comparable. In both cases the class being sorted implements an interface with a single method that compares the current object (this) with another object, and the sorting algorithm depends only on that interface. Sorter plays the role of Collections.sort: it sorts any collection of objects that implement the interface without knowing their concrete type.

There are three differences. `compareTo` returns an int (negative, zero or positive), so it can also express that two objects are equal, while `isBigger` only returns a boolean. Comparable is generic (Comparable<Person>), so the compiler checks the type and no cast is needed. Finally, Java already provides the sorting algorithm in Collections.sort, so the programmer only has to define the order.

10. Implement your program to sort Persons and Rectangles by using the Comparable interface and the Collections class.

Java already provides this mechanism. In v3, Person implements Comparable<Person> and Rectangle implements Comparable<Rectangle>. Each class defines its natural order in compareTo, which compares the current object with the argument.

```
// In Person implements Comparable<Person>
public int compareTo(Person other) {
    int c = surname.compareTo(other.surname);
    if (c != 0) {
        return c;
    }
    return name.compareTo(other.name);
}

// In Rectangle implements Comparable<Rectangle>
public int compareTo(Rectangle other) {
    return Double.compare(this.area(), other.area());
}
```

compareTo returns an int instead of a boolean:

Return value	Meaning
Negative	The current object goes before the other object.
Zero	Both objects have the same sort key.
Positive	The current object goes after the other object.

Main stores the objects in a List and sorts them with Collections.sort, which accepts any list whose elements implement Comparable. The custom Sorter and Sortable are no longer needed.

```
Collections.sort(people);
Collections.sort(rects);
```

v3.Main produces the same output as v2.Main. Two differences with the custom version are worth noting. First, generics (Comparable<Person>) remove the explicit cast: the compiler checks the type. Second, Double.compare is used for the areas because subtracting two doubles and converting the result to int would round small differences to zero.